

LLD Practice Platform

System Architecture, Domain Design & Technical Specification

Target Assignment: CipherSchools 2-Day Engineering Challenge	Date: September 2026
Candidate / Author: Rahul (akcodes-py)	Version: 1.0 (Production Candidate)
Repository: github.com/akcodes-py/LLD--practice-Platform	Tech Stack: React + TS + Django + PostgreSQL

1. Executive Summary & Problem Framing

Low-Level Design (LLD) practice is critically underserved by existing prep resources. While algorithms have deterministic judges (LeetCode) and courses offer passive video walkthroughs, LLD practice suffers from four core cognitive hurdles:

- **No Single Ground Truth:** In LLD, two competent designs (e.g. central coordinator vs distributed actors) can look entirely different. Learners get paralyzed asking *'Which answer is correct?'* rather than evaluating trade-offs.
- **Vague, Unactionable Feedback:** Human feedback is slow or absent; naive AI prompts drift wildly without fixed rubrics, often praising bad designs or hallucinating arbitrary scores.
- **Lost Iteration Evidence:** Designs live in ephemeral whiteboards or notebooks. Progress is invisible because attempts are never preserved across a learning loop.
- **Ambiguous Submission Bar:** Learners rarely know what a complete design specification requires (requirements mapping, class contracts, relationships, edge cases, trade-offs).

Core Product Goal: Build the smallest submission format that reliably proves design thinking, evaluated by the smallest feedback engine that truly teaches.

2. MVP Scope & End-to-End User Journey

The platform delivers an unhindered practice loop without bloatware (no LMS, payments, or mandatory auth):

Catalog Selection (/problems) → Problem Brief (/problems/:slug) → Drafting Attempt (/attempts/:id with autosave & validation) → Async Submit (202 Accepted + background worker) → Rubric Feedback Review (/attempts/:id/review) → Attempt History (/history) → One-Click Retry (clones to new draft)

5 Included Interview-Grade Problems:

Problem	Key Design Complexity & Architectural Patterns	Est. Time
Parking Lot	Multi-floor vehicle sizing, dynamic spot allocation (Strategy pattern), barrier gates, concurrency locks.	45 min
Elevator System	Dispatcher algorithms (LOOK/SCAN), internal/external requests, motion state machine, emergency handling.	45 min
Vending Machine	Explicit State Machine (Idle, HasMoney, Dispensing, SoldOut), coin/cash validation, inventory locks.	30 min
Library Management	Catalog indexing, fine computation rules (Strategy), hold/reservation queue, borrowing limits.	30 min
Food Ordering System	Restaurant menus, multi-item cart lifecycle, payment gateway integration (Adapter), live order tracking.	45 min

3. System Architecture & Domain Model

The architecture is structured as a resilient, modular monolith. The system enforces strict separation of concerns: views remain thin HTTP translation layers, all validation and state transitions live in `services.py`, and evaluation engines comply with a shared result schema.

```

Frontend (React 18 SPA)
■■■ Pages: Home, ProblemList, ProblemDetail, AttemptEditor, AttemptReview, History
■■■ Lib: REST API Client (Fetch + Poller), Design Tokens, Metadata Hooks
■
▼ REST API (JSON Over HTTP / Reverse Proxy)
Backend (Django 5 + Django REST Framework)
■■■ HTTP Layer (practice/views.py): Thin endpoints, standard error envelope
■■■ Domain Layer (practice/services.py): Content validation, async runner, retry/clone
■■■ Data Entities (practice/models.py): Problem, Attempt, Evaluation
■■■ Evaluators (practice/evaluators/):
■■■ base.py - Evaluation result contract & schema validation
■■■ rule_based.py - Deterministic heuristic scorer + citation extractor
■■■ llm.py - Optional OpenAI-compatible schema-constrained evaluator
■
▼ Django ORM (WAL Journaling + Concurrency Retry Handlers)
Storage: SQLite (Local Zero-Config) / PostgreSQL 16 (Docker Production)

```

Domain Entities & Class Responsibilities:

Entity / Module	Single Responsibility	Invariants & Dependencies
Problem	Immutable specification prompt (requirements, constraints, edge cases, suggested time).	Zero dependencies. Seeded idempotently via <code>seed_problems</code> .
Attempt	Encapsulates the 6 design sections, learner status (draft, submitted, evaluating, completed, failed), and timestamps.	Belongs to a Problem. Edits permitted only when in draft state.
Evaluation	Stores immutable assessment payload (provider, total score, 8 rubric criteria items, overall feedback, cited evidence).	1-to-1 relationship with Attempt. Conforms strictly to <code>base.py</code> contract.
services.py	Orchestrates attempt creation, patch autosaving, synchronous persistence, async worker dispatch, and retry cloning.	Coordinates models and evaluators. Ensures DB commit precedes thread spawn.
evaluators/base.py	Defines the formal evaluation contract (<code>validate_result</code>) ensuring provider swappability.	No external dependencies. Validates types, bounds (0-10), and mandatory quotes.

4. Submission Format & Evaluation Engine

Why Structured Text over Code or Diagrams?

Full code editors necessitate language runtimes and compiler sandboxes; diagram canvases burden learners with UI manipulation. Neither isolates architectural judgment. The six-section structured text format requires the candidate to explicitly communicate responsibilities, interface boundaries, and trade-offs—the exact signal evaluated in LLD interviews.

The 8-Criterion Explainable Rubric:

#	Rubric Criterion	What Is Evaluated & Evidence Extracted
1	Requirement Understanding	Mapping of functional requirements into domain entities; clarity of operational assumptions.
2	Responsibilities & Cohesion	Single Responsibility Principle (SRP); avoidance of monolithic 'God' classes.
3	Coupling & Class Relationships	Dependency direction, composition over inheritance, avoidance of tight cyclic couplings.
4	Encapsulation & Interfaces	Public contracts vs private state; adherence to Interface Segregation & Dependency Inversion.
5	Abstraction & Patterns	Judicious use of design patterns (Strategy, State, Observer, Factory) without over-engineering.
6	Extensibility & Trade-offs	Open-Closed Principle (OCP); clear justification for chosen architectural trade-offs.
7	Edge Cases & Testability	Handling of concurrency, hardware/network failure, capacity limits, and unit test strategy.
8	Explanation Quality	Communication clarity, coherent workflow execution traces, and structured reasoning.

5. Deterministic vs. Judgment Split & Resilience

A foundational principle of this architecture is separating deterministic boundaries from subjective judgment:

- **Deterministic Pipeline:** Input validation (min 60 chars per section), honeypot spam detection, idempotency key deduplication, and lifecycle state enforcement are 100% deterministic.
- **Rule-Based Evaluator (Default):** Executes completely offline in <10ms. Analyzes structural breadth, vocabulary, pattern markers, and quotes candidate text. Always labelled truthfully as '*Rule-based review*' in the UI.
- **LLM Evaluator (Optional):** Activated only via LLM_ENABLED=1. Bound by strict system prompts, JSON output schemas, and 20s timeouts. If the LLM produces invalid JSON or fails, the attempt transitions to a visible failed state with an instant *Re-evaluate* action.

State Machine & Failure Transitions:

```
[DRAFT] ■■(Submit Request)■■■ Persist synchronously to DB
■
▼
Spawn Daemon Evaluation Thread
■
[EVALUATING] (Client polls every 1.5s)
■ ■
(Evaluator Success) (Timeout / Parser Error)
▼ ▼
[COMPLETED] [FAILED]
■ ■
▼ ▼
One-Click Retry Re-Evaluate Action
(Clones into fresh Draft) (Re-triggers async worker)
```

Crash Safety Guarantee:

In services.py, the submission content is atomically written to disk and status set to evaluating *before* the thread is launched. Even if the backend process suffers a hardware restart or crash mid-evaluation, the user's design is never lost. On boot, the user sees the attempt and can trigger re-evaluation.

6. REST API Specification & Error Handling

All endpoints reside under /api/. Standard errors share a uniform envelope: {"error": {"code", "message", "details"}}.

Method & Route	Description & Request Payload	Status Codes
GET /api/health/	Database liveness and container health check.	200 OK
GET /api/problems/	Lists all 5 problems with metadata, difficulty, tags, and summary.	200 OK
GET /api/problems/<slug>/	Full brief: functional requirements, constraints, edge cases.	200, 404
POST /api/attempts/	Creates draft attempt. Accepts problem_slug & idempotency_key.	201, 200 (dedup)
PATCH /api/attempts/<id>/	Autosaves draft sections. Enforces draft-only lock.	200, 409 Conflict
POST /api/attempts/<id>/submit/	Validates fields, commits row, dispatches background evaluation.	202 Accepted, 400
GET /api/attempts/<id>/status/	Lightweight poller returning status and evaluation result if complete.	200 OK, 404
POST /api/attempts/<id>/retry/	Clones content into fresh draft (history preserved).	201 Created
POST /api/attempts/<id>/re-evaluate/	Re-runs evaluator on failed attempts.	202 Accepted, 409

7. Architectural Change Tests & Trade-offs

A great design is evaluated by how easily it accommodates future change without rewriting core abstractions:

Change Test A: Adding a Diagram Submission Format

- *Requirement:* Support Mermaid.js or visual graph node/edge JSON submissions.
- *Impact on Current Design:* Add a nullable `diagram_data = JSONField()` to `Attempt`. The core state lifecycle (draft → submit → evaluate → retry) and problem catalog remain untouched. An adapter extracts class names and relationships from diagram nodes to feed the existing rubric contract.

Change Test B: Adding Human Mentorship Review

- *Requirement:* Allow human senior engineers to grade attempts alongside automated review.
- *Impact on Current Design:* The `Evaluation` entity already accepts arbitrary provider keys (e.g. 'human_mentor'). A mentor review UI simply posts to an internal evaluation endpoint with the identical 8-rubric JSON schema. The frontend review component renders it transparently.

Scalability Trade-off Analysis:

Dimension	Choice in MVP	Production Scale Step (Roadmap)
Async Worker	Python daemon thread + DB pooling. Zero broker infra for local dev.	Persistent <code>EvaluationJob</code> table polled by dedicated worker, or Celery + Redis.
Database	SQLite with WAL + busy timeout (zero-setup dev); Postgres in Docker.	Managed PostgreSQL with read-replicas for catalog and history queries.
Authentication	Omitted for 2-day MVP to focus 100% on the core LLD practice loop.	OAuth2 / Session auth partitioning <code>Attempt</code> records by <code>user_id</code> .

8. Quality & Pre-Launch Verification Summary

Audited against the mandatory 40-point verification checklist (see docs/CHECKLIST.md):

- **Website Quality (20/20):** Responsive layouts tested down to 360px (zero horizontal scroll), working mobile drawer, SVG favicon, per-route document titles and meta descriptions, accessible focus rings, current copyright year, custom 404 handler, and zero broken links.
- **Pre-Launch & Security (20/20):** Functional Privacy Policy and Terms of Service, secrets isolated strictly to backend environment variables (never in frontend code), Open Graph preview cards, valid robots.txt and sitemap.xml, and honeypot protection.
- **Test Suite Coverage:** 13 backend tests (Pytest) exercising full API journeys, input validation, duplicate rejections, and evaluator contracts; 4 frontend tests (Vitest) validating rubric formatting and router smoke.

Architectural Takeaway:

The LLD Practice Platform demonstrates that effective domain design is not about piling up distributed tools or decorative microservices, but about crafting cohesive, single-responsibility domain entities, clear interface contracts, honest evaluation feedback, and crash-resilient user loops.